

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Homework05 - UMLReading

Slides 1.01 01

Slides 1.05 07

Slides 1.05 08

Slides 1.05 09

Slides 1.05 10

Tocagon Perugachi Nohemi Ninibeth

October 14, 2026

DÍA MES AÑO NOTA 7

"Más se estima lo que con más trabajo se gana" Aristóteles

Object Oriented Programming (OOP)
OOP PRACTICES

SUMMARY

Conceptual Modeling helps developers represent real-world ideas and system in a clear way. It is used to understand a system, analyze its requirements, plan its design, and diagrams that describe different aspects of a software system. UML supports this process by providing the diagrams.

Lose Coupling means organizing software into layers with clearly defined responsabilities. Each layer uses the services or abstractions provided by other layers without depending too much on their details. Aggregation over inheritance is often preferred because objects can reuse other objects more flexibly. Programming to interfaces, choosing meaningful names and writing executables unit tests also improve software readability, flexibility and reliability.

Unified Modeling Language

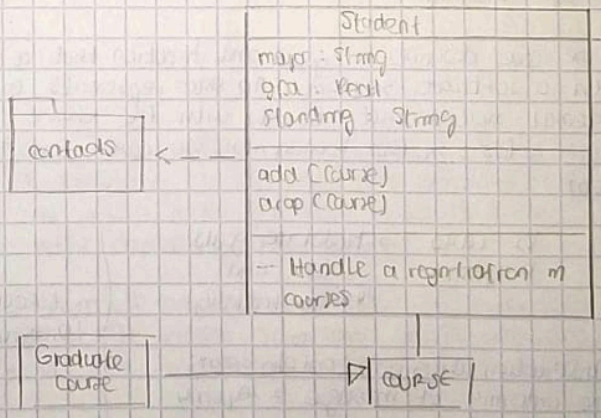
UML

UML is a standart modeling language used to visualize, specify, and document software systems. It includes different types of diagrams that represent the system's structure and behavior. Structural diagrams describes classes, objects, and relationships, with behavioral diagrams explain activities with interactions.

Diagram that communicate ideas using the modeling elements

Modeling elements are like words

Diagrams are like sentences and paragraphs



UML can be used throughout software development, from analysis to implementation. Views represent different perspectives of a system, such as use cases, logical structure, implementation, processes and deployment

Abstraction allow developers to focus on essential information without showing every technical detail.

Three Software C P.

1. Analysis (Problem Domain) Capture real world concepts, and business rules without software details

2. Specification (Solution Domain) Define logical software abstractions, interfaces, encapsulation and automation boundaries

3 Implementation (Construction) Focuses on concrete software components, data structures, libraries and physical deployment

Student
name major GPA standing interests
-- The set of students know the registration system

Classes define abstraction boundaries and encapsulations for software objects

Student
name : String major : String GPA : Real random : Integer
addCourse() dropCourse()
-- Software representation of students; supported operations on courses

Use Cases and Use Case Diagram

A use case describes a goal and function that a user wants to accomplish with a software system. An actor represents a person organization or external system that interacts with the software. use case diagrams show actors, system boundaries, use cases, and the relationships between them.

Use Cases Diagrams → Model user goals, actors and system boundaries

Interaction diagrams emphasizing the ordering of message → sequence of diagrams.

Communication: structural organization between objects

Statechart Diagrams → Model internal event-driven state transition of objects

The << include >> relationship represent behavior that is required or reused by another use case. In contrast << extend >> represents additional behavior that occurs under certain conditions. Generalization shows that a specialized actor or use case shares characteristics with a more general one. These relationships help describe system functionality clearly.

Objects, classes and
Class diagrams

An object is an individual entity with its own identity, state, and behavior. A class define a group of similar objects and describes their common characteristic.

Classes can contain attributes, which represent information, and operations with describes the action or service they provide.

Class diagram show classes and the relationships between them. Association represents a relationship between objects, while generalization represents a inheritance relationship also known as a "is-a" relationship.

Aggregation represents a whole-part relationship in wich the parts can exits independently. Composition is stronger because the parts depend on whole for their lifecycle. Multiplicity specifies how many objects can participate in a relationship, and dependency indicate that one element relies on another.

Activity
DIAGRAM

An activity diagram represents the workflow of a process, showing the sequence of actions from the beginning to the end. The initial node marks where the process starts and action nodes represent individual tasks. Decision nodes select different paths depending on a condition.

Fork and join node represent parallel activities and their synchronization. The final node mark the end of the process. Swim lanes divide the diagram according to the person, role, or component responsable for each activities.

These diagrams help developers understand the order of task and how responsabilities are distributed.

Interaction Diagram

Describes how objects communicate by exchanging message to accomplish a task. They show participants involved, the messages they send and the relationship between them.

A sequence diagram emphasizes the order in which messages occur over time. It commonly include objects, lifelines, and messages representing method calls or other communication. A communication diagram focuses more on the connections between objects and how they collaborate. Both diagram describes interations, but each emphasizes a different aspect of the same system behavior.

Messages can be synchronous or asynchronous. With a synchronous message the sender waits for the operation to be handled before continuing. With an asynchronous message, the sender can continue without waiting. Reply message represent responses, while create messages indicate that a new object is created.

State Charts

A state chart describes the different states an object can occupy and how it moves from one state to another in response to events. An initial state represents the starting point while a state represents a condition or mode of operation.

A transition connects states and may include a trigger, a guard condition, and an action, the trigger identifies the event that causes the transition the guard specifies a condition that must be true, and the action describe what happens during the transition. State charts are useful for understanding objects whose behavior changes depending on their current sates.

Main Differences to Remember

A class defines the characteristics of objects, while an object is a particular instance of a class. Attribute store information, whereas operations define behavior. Association describes a relationship between objects; aggregation and composition describe whole-part relationships with different levels of dependency.